

TeachLoop: An AI-Powered Communication Practice Platform for Technical Professionals

Cheng-I Wu
Independent Developer
Email: cheng-i@example.com

Abstract—Effective communication of technical concepts is a critical professional skill rarely taught in formal education. This paper presents TeachLoop, a web-based platform that helps engineers practice explaining complex technical topics through AI-powered feedback. The system employs a three-mode architecture (Teaching, Interview, Custom) with audience-aware evaluation across five communication dimensions. A progressive hint system provides scaffolding from gentle nudges to comprehensive explanations, drawing on Vygotsky’s Zone of Proximal Development. The platform uses a client-side state machine with localStorage persistence, proxying evaluations to a large language model via a single API route. We describe the system architecture, the AI evaluation pipeline with rate limiting and schema validation, the design system comprising three themes with 40+ CSS custom properties, and accessibility features targeting WCAG AAA compliance. A content database of 36 curated questions with 108 progressive hint sets covers AI/ML concepts and system design topics. TeachLoop demonstrates that effective AI-assisted communication practice can be delivered with zero backend infrastructure beyond the AI proxy.

Index Terms—communication practice, AI feedback, scaffolding, web application, technical education, progressive hints

I. Introduction

Software engineers frequently struggle to explain complex systems to non-technical stakeholders, junior developers, or interviewers. While technical knowledge is necessary, the ability to communicate that knowledge effectively determines career outcomes—from passing technical interviews to leading cross-functional projects. Studies in engineering education consistently show that communication skill is among the top factors in hiring decisions, yet it is rarely taught in formal curricula [1].

Existing tools focus on technical skill assessment—coding challenges, system design quizzes, flashcards—but neglect the communication dimension entirely. A developer may understand backpropagation deeply but be unable to explain it to a product manager. TeachLoop addresses this gap by evaluating how users explain, not merely what they know.

The platform presents users with technical concepts (e.g., backpropagation, CAP theorem, transformer architecture, ride-sharing system design) and asks them to explain these concepts to various audiences—from a curious 10-year-old to a senior AI engineer. An AI model evaluates each explanation across five communication dimensions and provides actionable feedback. The system is designed

around four explicit principles: coach over critic, focus on the human, progress is visible, and warmth through craft.

The remainder of this paper is organized as follows. Section II reviews related work on scaffolding and AI-assisted feedback. Section III describes the system architecture, state machine, and AI evaluation pipeline. Section IV details the design system and theme architecture. Section V covers implementation details including code splitting, voice input, security, and accessibility. Section VI discusses trade-offs and limitations. Section VII concludes.

II. Related Work

A. Scaffolding in Education

Vygotsky’s Zone of Proximal Development (ZPD) establishes that learners achieve more with structured support than alone [2]. The ZPD is the gap between what a learner can do independently and what they can achieve with guidance. Hint systems in intelligent tutoring employ progressive revelation—from vague prompts to detailed explanations—to keep learners within their ZPD [3]. This scaffolding approach has been validated across domains from mathematics to programming education. TeachLoop adapts this pattern: each concept offers three hint levels (nudge, concept outline, full explanation), allowing users to self-select their scaffolding depth.

B. AI-Powered Feedback

Recent work on AI-assisted writing feedback demonstrates that large language models can provide structured, dimension-specific evaluation when given carefully engineered prompts [1]. TeachLoop extends this to oral and written technical explanations, scoring across clarity, analogies, vocabulary fit, confidence, and structure. The evaluation prompt is designed to produce consistent JSON output, enabling programmatic aggregation into session-level reports.

C. Spaced Repetition and Practice

Effective skill acquisition requires repeated practice with increasing difficulty. The Leitner system [4] formalizes spaced repetition through progressive intervals. While TeachLoop tracks session history and score trends to make progress visible, adaptive difficulty and spaced repetition remain future work. The current system uses random question selection from curated pools.

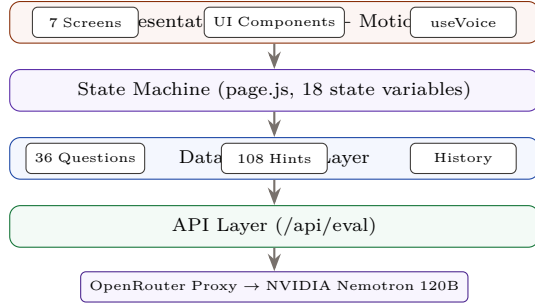


Fig. 1: Four-layer architecture of TeachLoop.

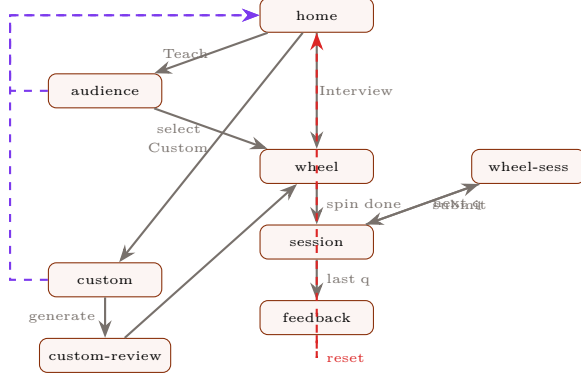


Fig. 2: Screen state machine with 8 screens and transition triggers.

III. System Design

A. Architecture Overview

TeachLoop follows a client-side-first architecture with four layers (Fig. 1). The design prioritizes simplicity: no backend database, no user authentication, no client-side router. All application state lives in a single React component; all persistent data lives in `localStorage`.

The Presentation layer contains seven screen components (Home, Audience, Wheel, Session, Feedback, CustomMode, CustomReview) with Motion-powered page transitions. The State Machine layer manages all application state via `useState` hooks in the root `AppInner` component—there is no client-side router; a screen state variable controls which screen is rendered. The Data & Hooks layer provides static question banks, progressive hints, voice input via the Web Speech API, and `localStorage` session persistence. The API layer consists of a single Next.js API route that proxies evaluation requests to OpenRouter.

B. Screen State Machine

The application operates as a finite state machine with 8 screens (Fig. 2). The screen state variable is the single source of truth for navigation state. There is no URL-based routing—the browser URL remains `/` throughout the entire session.

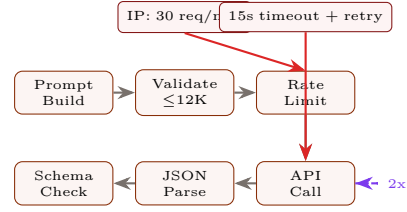


Fig. 3: AI evaluation pipeline with validation gates and retry logic.

The root component maintains 18 state variables that collectively encode the full application state. Key variables include: `screen` (current screen), `mode` (teach/interview/custom), `qs` (question array for current session), `idx` (current question index), `ans` (current answer text), `evals` (accumulated evaluation results), `report` (session-level summary), `wheelSpin` (animation state), and `wasVoiceUsed` (microphone flag). Session history is managed by the `useSessionHistory` hook, which persists up to 20 sessions in `localStorage`.

C. AI Evaluation Pipeline

Each user answer is evaluated by an LLM through a structured pipeline (Fig. 3). The pipeline implements five validation stages before reaching the AI model.

The prompt construction stage builds a mode-specific evaluation prompt. Teaching mode evaluates: clarity, analogies, vocabulary fit, confidence, and structure. Interview mode evaluates: clarity, depth, structure, confidence, and examples. Each dimension is scored 1–10. The prompt also requests structured JSON output containing an `overallScore`, `styleObservation` (one sentence), `strongPoint`, and `flawToFix`.

Input validation caps prompt length at 12,000 characters. Rate limiting enforces 30 requests per minute per IP address using an in-memory store. The API call uses a 15-second timeout via `AbortController` with exponential backoff retry (2 attempts). JSON extraction includes fallback parsing for malformed responses. Schema validation ensures the response matches the expected shape before returning to the client.

After all questions in a session are evaluated, a session report is generated by aggregating individual evaluations into a communication style analysis, recurring strength, recurring flaw, and a weekly practice tip. This report is saved to `localStorage` as part of the session history.

D. Progressive Hint System

Each of the 36 static concepts offers three hint levels following scaffolding theory [3]:

L1: Nudge: A gentle directional prompt that points toward the answer without revealing it. Example for “Attention Mechanism”: “Focus on the core idea of weighing how much each input word matters to every other word.”

TABLE I: Theme Variants with Accent Colors

Theme	Accent	Background
Sunrise Studio	Burnt Orange #C2410C	Warm cream #FBF9F6
Chalkboard	Yellow #FACC15	Dark zinc #18181B
Blue Hour	Blue #2563EB	Cool slate #F8FAFC

L2: Concept Outline: A structured scaffold providing key concepts and terminology. Example: “Attention lets each token look at all other tokens and decide which ones are most relevant. Explain query, key, value vectors and the softmax weighting.”

L3: Full Explanation: A comprehensive reference covering what a strong answer should include. Example: “In ‘Attention Is All You Need’, each token is projected into Q, K, V. Attention scores = $\text{softmax}(QK^T/\sqrt{d})V$. Multi-head attention runs this in parallel.”

The content database covers 20 AI/ML teaching concepts (from Backpropagation to Mixture of Experts) and 16 system design/SWE interview questions (from URL shortening to consensus algorithms). Each hint set is hand-curated to ensure technical accuracy and pedagogical progression.

E. Voice Input Pipeline

The useVoice hook captures speech via the Web Speech API and applies automatic post-processing. Filler words (“um,” “uh,” “like,” “you know,” “I mean,” “basically,” “literally,” “right”) are removed. Consecutive duplicate words are collapsed. Punctuation is normalized. A wasVoiceUsed flag annotates the evaluation prompt to excuse transcription artifacts: “Grade the ideas and communication, not the transcription quality.”

IV. Design System

A. Typography and Spacing

The typography follows a Perfect Fourth (1.333) modular scale with 10 defined steps from 0.6875rem (2xs) to 3.375rem (5xl). The primary typeface is Geist, a variable font optimized for screen reading. Spacing uses a 4pt base with 16 defined steps from 0.25rem to 8rem. Border radii range from 0.25rem (xs) to 9999px (full). Easing curves include quart, expo, spring, and in-out variants. A z-index scale from 0 (base) to 500 (toast) prevents stacking conflicts.

B. Theme System

Three themes are implemented via CSS custom properties, each defining 40+ variables for colors, shadows, grain texture, glass effects, and wheel segment colors (Table I).

Theme selection persists in localStorage and is applied via the data-theme attribute on the document root. An inline script in the root layout runs before React hydration to prevent flash-of-unstyled-theme (FOUC). The ThemeProvider context exposes the current theme and a switcher component (three colored dots with spring-scale animation).

TABLE II: Technology Stack

Component	Technology & Rationale
Framework	Next.js 15.3.3 (App Router, API routes, React Server Components)
UI	React 19.1.0 (concurrent features, Suspense for code splitting)
Animation	Motion 12.42.0 (spring physics, AnimatePresence)
Typography	Geist 1.7.2 (variable font, screen-optimized)
Icons	Phosphor Icons 2.1.10 (duotone weight, consistent language)
AI Backend	OpenRouter / NVIDIA Nemotron 120B (free tier, JSON mode)
Speech	Web Speech API (native browser, no dependencies)
Storage	localStorage (max 20 sessions, JSON serialization)
Deployment	Vercel (edge functions, zero-config Next.js)

C. Visual Effects

The design system employs seven layered visual effects:

- (1) Grain Texture: An SVG noise overlay at 2–4% opacity is applied via a ::after pseudo-element on the body, adding tactile warmth without impacting readability.
- (2) Glassmorphism: The navigation bar uses backdrop-filter: blur(16px) with a semi-transparent background, creating a depth hierarchy between the nav and page content.
- (3) ScoreRing: An SVG circle with animated stroke-dashoffset represents scores. Color coding: green for scores ≥ 8 , accent for 5–7, red for < 5 . Configurable size and animation delay.
- (4) Sparkline Charts: SVG polylines render score trends across past sessions. Auto-scales to min/max range. Up to 5 sessions are shown on the home screen.
- (5) Wheel Animation: A CSS conic-gradient wheel with segment labels spins using quadratic easing (3–5 extra rotations). A glow effect activates during spin. Progress dots with connecting lines track completed questions.
- (6) Page Transitions: AnimatePresence with 250ms opacity fade between all screens.
- (7) Staggered Entrances: Audience list items animate in with 50ms stagger delay for a polished reveal effect.

V. Implementation

A. Technology Stack

Table II summarizes the technology stack and the rationale for each choice.

B. Code Splitting and Dynamic Imports

Heavy screen components (AudienceScreen, WheelScreen, SessionScreen, FeedbackScreen) use Next.js dynamic() imports with no loading placeholder. The app shell (HomeScreen, CustomModeScreen) loads eagerly. This reduces the initial bundle from the full application to just the entry screen and shared utilities.

C. Error Handling

A class-based `ErrorBoundary` component wraps the entire application, catching React rendering errors and providing a recovery UI with a “Try again” button. API errors are caught at the call site and displayed in contextual error banners with retry options. The session report generation has independent error handling—if report generation fails, users can still view their individual answers.

D. Security

The API route implements multiple security layers: IP-based rate limiting (30 requests/minute), origin validation against allowed origins, prompt length capping (12,000 characters), 15-second fetch timeout with `AbortController`, exponential backoff retry (2 attempts maximum), JSON extraction with regex fallback for malformed responses, and response schema validation. Security headers include `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, and `Permissions-Policy` with `microphone=self`.

E. Accessibility and PWA

The platform targets WCAG AAA compliance with: a skip link to `#main-content`, screen reader text via `.sr-only` class, `:focus-visible` outlines with accent color, `prefers-reduced-motion` media query that disables all animations, semantic HTML with proper heading hierarchy, and full keyboard navigability.

PWA features include a web manifest with app name and theme color, a service worker implementing stale-while-revalidate caching, theme-color meta tag for mobile browser chrome, and a `beforeunload` handler to prevent accidental navigation during active sessions.

F. Data Architecture

All data is client-side. Static content (36 questions, 108 hint sets, 5 audience definitions) is bundled as JavaScript modules. Session history (date, mode, audience, overall score, per-question scores) persists in `localStorage` with a 20-session cap. Theme preference and onboarding dismissal state also persist. There is no server-side database, no user authentication, and no cross-device synchronization.

VI. Discussion

A. Design Trade-offs

Several deliberate trade-offs were made during development:

- (I) No client-side router: Using a screen state variable instead of URL-based navigation simplifies state management but prevents deep linking, browser back-button support, and shareable URLs for specific screens.
- (II) Single-component state: All 18 state variables live in `AppInner`, avoiding prop drilling complexity but

creating a large component (~357 lines) that re-renders on any state change.

- (III) Static content database: Embedding questions in the bundle ensures instant availability but limits the question pool to 36 static items. Custom mode mitigates this by generating questions dynamically.
- (IV) `localStorage` only: No backend complexity, but session history is device-bound and lost on cache clear. No cross-device sync is possible.
- (V) Free AI model: Using NVIDIA Nemotron 120B via OpenRouter’s free tier eliminates cost but introduces potential rate limits and model availability risks.

B. Limitations

The current implementation has several limitations. No user accounts or cross-device synchronization. No adaptive difficulty—question selection is random rather than performance-based. No exportable session reports beyond the browser print dialog. Custom mode requires AI generation, adding latency and an external API dependency. Voice input accuracy varies across browsers and hardware.

C. Future Directions

Planned improvements include: adaptive question selection using evaluation scores to target weak areas, spaced repetition scheduling for concept review, multi-language support for global audiences, team practice mode with shared evaluation reports, and enhanced PWA features including background sync and push notifications for practice reminders.

VII. Conclusion

TeachLoop demonstrates that effective AI-assisted communication practice can be delivered with a minimal, client-side-first architecture. By combining a curated content database (36 questions, 108 hint sets), structured AI evaluation (5-dimension scoring), and a craft-oriented design system (3 themes, Perfect Fourth typography, grain textures), the platform provides actionable feedback on how engineers explain technical concepts. The key architectural decision—a single API route proxying to a free LLM with `localStorage` persistence—eliminates backend infrastructure while maintaining a polished user experience. The progressive hint system, grounded in scaffolding theory, provides entry points for users at different skill levels. Future work will focus on adaptive difficulty and spaced repetition to improve long-term learning outcomes.

References

- [1] DeepLearning.AI, “Communicating with AI: Technical Explanations,” DeepLearning.AI Short Courses, 2024. [Online]. Available: <https://www.deeplearning.ai>
- [2] L.S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA: Harvard Univ. Press, 1978.

- [3] K. vanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.
- [4] S. Leitner, *So lernt man lernen*. Freiburg: Herder, 1972.